

RapidBoxForest: Link-Envelope-Guided Box Forests for Low-Overhead Scene Construction in Multi-Query Manipulation

Yu Tian and Hongliang Ren

Abstract—Repeated-query manipulation needs reusable free-space structure, but certified convex regions can be expensive to construct and update in cluttered workcells. RapidBoxForest (RBF) validates axis-aligned joint boxes with endpoint-to-link swept-capsule bounds, organizes accepted boxes as a scene-level forest and typed corridor graph, and stores kinematics-keyed envelope evidence in a Lifelong Envelope Cache Tree (LECT) separately from scene labels. Certificates are conditional: conservative box or corridor unions are covered, whereas repaired, shortened, or filter-based paths require query validation. Saved-catalog experiments use audit-separated timing and explicit cache accounting. On Shelf+IIWA, warm depth-23 LECT replay builds the registered structure in 0.166 s, with 0.130 s per-query online time and 0.164 s five-query amortized time; live hierarchical interval forward kinematics (HIFK-5) evidence materialization raises build time to 7.006 s. At registered saved-random-scene operating points, RBF has lower per-query online time than Batch Informed Trees (BIT*) checkpoint rows in four of six robot-difficulty panels. In a maintenance-only study, leaf-sweep insertions are 153.3–207.5× faster and removals are 943.6–3354.0× faster than the 15-obstacle warm rebuild. Snapshot creation, loading, resident memory, and replanning after updates are excluded from these timing rows.

Index Terms—Motion planning, multi-query planning, link interval envelopes, box-region planning, configuration-space planning.

I. Introduction

Many high-degree-of-freedom manipulation workloads contain related queries in nearly fixed workcells. Robot kinematics persist, while start-goal pairs, local obstacles, or small scene edits change. Convex-region planners, including IRIS (Iterative Regional Inflation) and Graphs of Convex Sets (GCS), provide optimization-ready region-level abstractions once regions are certified [1], [2], [3], but region construction can dominate in high-dimensional or frequently edited scenes. Sampling-based planners, including PRM and RRT variants, avoid such region construction [4], [5], [6], [7], but their reusable structures are usually zero-volume roadmaps or query trees.

RBF targets this gap with a lower-overhead volumetric model: it grows axis-aligned joint-space boxes, validates them with link-envelope bounds, connects accepted boxes

into a typed corridor graph, and reuses that graph across later queries. The resulting trade-off is explicit: boxes are less expressive than rich convex covers, but they simplify validation, graph building, updates, and multi-query reuse.

a) Problem statement: Let $Q \subset \mathbb{R}^n$ be the joint-limit box of a fixed manipulator and let $\mathcal{O} \subset \mathbb{R}^3$ be the current workspace obstacle set. Given a set or stream of queries $(q_s, q_g) \in Q^2$, RBF builds a finite family of validated C-space boxes and an adjacency graph over those boxes. The design goal is low scene-construction and maintenance overhead while retaining reusable volumetric free-space evidence.

The geometric core adapts classical endpoint bounding. For rounded links, endpoint envelopes bound endpoint trajectories over a joint box; their radius-expanded convex hull encloses the swept link and validates C-space boxes. Conservative sources yield conditional box certificates, whereas tighter non-certifying sources remain filters whose paths require query validation. Reuse is split accordingly: scene-level RBF caches store one scene’s box forest and corridor graph, while the Lifelong Envelope Cache Tree (LECT) stores replayable kinematics-keyed endpoint and link evidence.

The evaluation preserves this boundary. Success is counted only after final query validation unless a row is explicitly conservative. Warm Shelf timings include replay-based scene construction and per-query retrieval; snapshot creation, loading, resident memory, and offline setup amortization are excluded. Dynamic update rows measure scene-cache maintenance, not end-to-end query pipelines. Thus the experiments evaluate low-overhead reusable-region construction under a stated validation policy, not dominance over single-query planners, repair-free certification, or completeness. This work contributes:

- 1) *Box validation:* conservative endpoint-to-link envelopes validate boxes; staged sources remain query-validated filters.
- 2) *Planning object:* validated boxes form a reusable scene forest and typed corridor graph.
- 3) *Two-level reuse:* scene caches hold boxes and graph state; LECT replays kinematics-keyed envelope evidence.
- 4) *Local maintenance:* deletions promote blocked domains; insertions trigger bounded refill.
- 5) *Evaluation protocol:* saved catalogs, audit-separated timing, and cache accounting define the reported scope.

The authors are with the Department of Electronic Engineering, The Chinese University of Hong Kong (CUHK), Hong Kong SAR, China. E-mail: ty1997@link.cuhk.edu.hk.

TABLE I
Qualitative taxonomy of reusable manipulation-planning paradigms. The comparison emphasizes the persistent object, validation granularity, and scene-edit semantics rather than ranking planners by runtime.

Family	Persistent object	Vol.	Validation and certificate unit	Scene-edit reuse	Main trade-off
PRM and LazyPRM	Roadmap vertices and edges	No	Local edges or path segments	Topology can be reused; affected edges require rechecking	Reusable multi-query graph, but reusable evidence is zero-volume.
RRT-Connect and RRT	Query tree	No	Incremental path segments	Little persistent reuse across queries	Query-focused discovery; persistent structure is query-local.
BIT* and FMT*	Batch sample graph or tree	No	Candidate graph edges	Primarily query-centric; limited scene-level reuse	Anytime quality improvement, but still edge-centric.
Adaptive cell decomposition	Configuration-space cells	Yes	Cells under exact, sampled, or interval tests	Explicit cells can be locally relabeled when maintained	Volumetric representation; scalability depends on the cell validator.
IRIS and GCS	Certified convex regions and graph	Yes	Convex free-space regions	Region reuse is scene-coupled; edits may require reflation or recertification	Optimization-ready regions with construction and recertification cost.
RBF	Box forest + typed graph + LECT evidence	Yes	Joint boxes via link envelopes; final audit when required	Scene boxes are maintained locally; LECT replays kinematics-keyed evidence	Lower-overhead volumetric reuse with less expressive boxes and conditional certificates.

Abbrev.: PRM=probabilistic roadmap; RRT=rapidly exploring random tree; BIT*=Batch Informed Trees; FMT*=Fast Marching Tree; IRIS=Iterative Regional Inflation; GCS=Graphs of Convex Sets; LECT=Lifelong Envelope Cache Tree; Vol.=volumetric.

Sections III–VI cover link envelopes, LECT, forest construction and updates, and evaluation.

II. Related Work

Convex-region planners construct certified free-space covers and search or optimize on the induced graph. IRIS variants inflate collision-free convex regions, and Graphs of Convex Sets (GCS) make the resulting graph optimization explicit [1], [2], [3], [8], [9], [10]. RBF shares the region-level objective but uses axis-aligned C-space boxes; this restricted geometry limits expressivity while keeping validation, adjacency, and update operations local and explicit. Table I summarizes the qualitative position.

a) Continuous-collision-detection endpoint bounds:

Swept-volume approximation and interval forward kinematics supply the geometric primitives used here. Convex-hull generator bounds, capsule radius lifts, the Proximity Query Package (PQP), Gilbert–Johnson–Keerthi (GJK), the Flexible Collision Library (FCL), and interval Denavit–Hartenberg propagation all support swept-sphere or endpoint-bounded collision reasoning [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22]. Redon *et al.* apply this pipeline to articulated-body continuous collision detection over time steps [23], [24]; related Taylor-model and safety methods use similar swept-sphere bounds [25], [26], [27]. RBF uses joint-box intervals rather than trajectory time steps as the bounding domain, stores compatible interval evidence in LECT, and assembles accepted boxes into a typed corridor graph. Repaired or post-processed paths remain subject to query validation.

b) Reuse and adaptive decomposition:

Multi-query planners amortize effort through reusable graphs. Probabilistic roadmaps (PRMs) and LazyPRM reuse roadmaps and edge checks [6], [28], [29], [30], experience-based systems reuse prior paths [31], and adaptive cell methods split cells until the accepted-cell graph can be searched [32], [33]. RBF combines these ideas with box-level envelope validation and a separate kinematics-keyed evidence cache.

c) *Sampling baselines and refinement:* RRT*, RRT-Connect, Batch Informed Trees (BIT*), Fast Marching Tree (FMT*), and the Open Motion Planning Library (OMPL) provide standard single-query or anytime implementations and benchmarking infrastructure [7], [34], [35], [36], [37], [38], [39]. Trajectory optimizers refine supplied paths [40], [41], [42], [43]. RBF is complementary: it supplies reusable volumetric context before repair, smoothing, and final validation.

III. Link Interval Envelopes

The link-envelope validation layer maps joint regions to endpoint and link bounds. For a joint region $I \subseteq \mathcal{Q}$, endpoint envelopes $E_k(I)$ bound endpoint trajectories and link envelopes $B_\ell(I)$ bound rounded links over the same region:

$$I \mapsto \{E_k(I)\} \mapsto \{B_\ell(I)\}.$$

Axis-aligned joint boxes are the default regions; local oriented bounding boxes (OBBs) in scaled joint coordinates are used only for typed bridge corridors. Conservative endpoint evidence gives certificates. Empirical or tighter performance-oriented endpoint sources are filters and must be followed by query validation.

Assumption 1 (Collision model for certificates). *Certificates cover configured active rounded links against $\mathcal{O}$, with joint limits enforced by $I \subseteq \mathcal{Q}$. Other contacts or constraints (self-collision, attachments, grippers, task regions) are outside this certificate unless encoded in the validation policy and cache descriptor; experiments check only active-link-workspace obstacles.*

A. Geometric Foundation

Let $T(q)x = R(q)x + t(q)$ be the transform at configuration q , and let $\mathcal{S}_I(C_0)$ denote the workspace set swept by body C_0 as q ranges over I . For a capsule $C = S \oplus B_2(r)$, the fixed radius ball commutes with rigid motion:

$$\mathcal{S}_I(C) = \mathcal{S}_I(S) \oplus B_2(r).$$

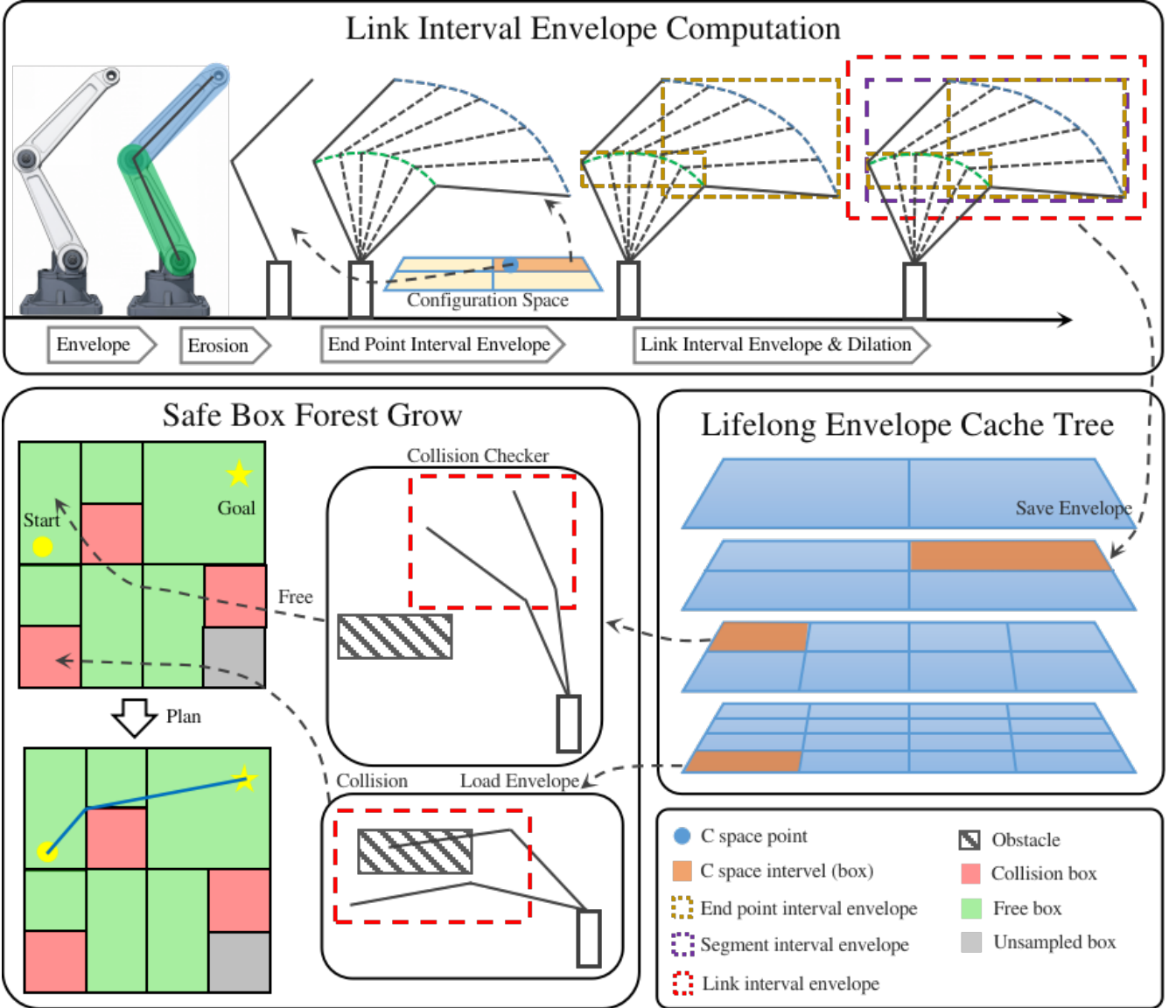


Fig. 1. RBF pipeline for link-envelope-guided multi-query planning. Top: endpoint interval envelopes induce link interval envelopes over a configuration-space (C-space) box. Middle: LECT reuses kinematics-keyed endpoint and link-envelope records across repeated builds. Bottom: scene filtering and sample-guided growth expand a connected scene-level box forest for repeated corridor queries.

Thus a rounded link can be bounded by first bounding the zero-radius center segment and then lifting by the link radius. For a convex body $P = \text{conv}(v_1, \dots, v_m)$, outer bounds on the generator trajectories also bound the swept body,

$$S_I(P) \subseteq \text{conv}\left(\bigcup_i \hat{\Gamma}_i(I)\right).$$

For a capsule link, only the two endpoint trajectories are needed. Let $\Gamma_a(I)$ and $\Gamma_b(I)$ denote the exact workspace trajectories of the proximal and distal endpoints over I . RBF uses these classical swept-sphere facts over a joint-box domain and stores the result as reusable planning evidence.

Proposition 1 (Endpoint envelopes induce link interval envelopes). *Consider a rounded link $L = [a, b] \oplus B_2(r)$ and a joint interval I . If endpoint envelopes $E_a(I)$ and $E_b(I)$*

satisfy $\Gamma_a(I) \subseteq E_a(I)$ and $\Gamma_b(I) \subseteq E_b(I)$, then

$$S_I(L) \subseteq \text{conv}(E_a(I) \cup E_b(I)) \oplus B_2(r).$$

Consequently, any outer representation of the right-hand side is a valid link interval envelope for the whole joint box I .

Proof. For each $q \in I$, the center segment of the moved link is the convex hull of the two moved endpoints, so $T(q)[a, b] \subseteq \text{conv}(\Gamma_a(I) \cup \Gamma_b(I))$ over the entire interval. The endpoint inclusions place this set inside $\text{conv}(E_a(I) \cup E_b(I))$. Adding the fixed radius ball commutes with the rigid-motion sweep of the capsule, giving the stated containment. $\square$

Corollary 1 (Conservative joint-box validation). *Let $I \subseteq \mathcal{Q}$ be a joint box and let $\{B_\ell(I)\}$ be conservative link interval envelopes for all active rounded links. If $B_\ell(I) \cap \mathcal{O} = \emptyset$*

for every link ℓ , then every configuration $q \in I$ is collision-free with respect to the collision model in Assumption 1.

Proof. By Proposition 1, each conservative link envelope contains the swept workspace volume of its link over all configurations in I . Disjointness from $\mathcal{O}$ for every active link therefore excludes workspace obstacle collision for the whole manipulator at every $q \in I$ under Assumption 1. $\square$

a) *Numerical conservatism:* A certificate is attached only when endpoint propagation, radius padding, and obstacle-disjointness tests are outward bounds under Assumption 1. Axis-aligned bounding box (AABB) padding uses a conservative B_∞ radius lift. SupportHull/GJK is certificate-backed only with outward obstacle inflation, margins, and termination tolerances. Empirical endpoint sources or post-processing outside the conservative box or OBB-corridor union are filters and do not extend Corollary 1 or Theorem 1.

B. Endpoint Interval Envelopes

For endpoint k , let $\Gamma_k(I)$ be its exact trajectory over interval I . Any conservative set containing $\Gamma_k(I)$ is an endpoint interval envelope; the basic record is an endpoint AABB because it is compact and directly reusable by all link-envelope representations. The planner uses two certificate-backed sources. Interval forward kinematics (IFK) propagates affine-arithmetic bounds along the Denavit–Hartenberg chain; shared noise symbols preserve some trigonometric correlation, making it a low-latency certificate-backed endpoint source in this implementation. Hierarchical IFK (HIFK) bisects the joint interval and hulls child endpoint boxes, reducing wrapping at higher materialization cost. IFK is the unsplit affine-arithmetic pass; HIFK is a true hierarchical split-and-hull evaluation (for example, HIFK-5 means five split levels), so the split depth is part of the cache descriptor and cannot be exchanged with an unsplit record. A critical-sample source probes interval endpoints, midpoints, near-boundary configurations, and trigonometric turning candidates; it can tighten boxes that IFK wraps, but does not certify coupled extrema. An analytical extrema source solves low-dimensional sinusoidal boundary subproblems, reducing one- and two-joint boundary cases to structured root search, and uses interval closure only when a conservative bound is needed. These two tighter sources serve as diagnostics or filters: any path depending on them is accepted only after final query validation unless wrapped by an independent conservative certificate. During tree descent, transient prefix transforms are reused to avoid recomputing unaffected chain prefixes; this is a runtime materialization optimization rather than persistent scene state. Fig. 1 shows the endpoint-to-link pipeline, LECT reuse, and box-forest use.

C. C-space OBB Endpoint Envelopes

Axis-aligned boxes remain the global forest primitive. Bridge repair motions can be joint-correlated, so RBF

may validate a local typed C-space OBB corridor without replacing the dyadic forest. OBBs are defined in scaled joint coordinates. Let $y = S(q - q_{\text{ref}})$, where S is a diagonal joint scaling, and let

$$R_{\text{obb}} = \{q_c + A\xi \mid \xi \in \Xi\},$$

$$\Xi = \prod_{i=1}^m [-1, 1], \quad A = S^{-1}U \text{diag}(h),$$

with U orthonormal in the scaled metric and h the halfwidth vector. The object R_{obb} is a configuration-space region, not a workspace OBB: the certificate comes from endpoint and link envelopes over R_{obb} , not from the fitted orientation.

a) *Cover certificate:* A conservative proof tiles the local cube, maps each tile to joint space, and validates the resulting AABB hulls:

$$I_r = \text{bbox}_q(q_c + AX_r).$$

If every I_r passes the standard conservative link-envelope scene test, then

$$R_{\text{obb}} \subseteq \bigcup_r I_r \subseteq \mathcal{C}_{\text{free}},$$

where $\mathcal{C}_{\text{free}}$ is the collision-free configuration set under Assumption 1. The stored OBB certificate is then a finite cover of ordinary box certificates.

b) *Correlated endpoint envelope:* A tighter source keeps the local OBB variables ξ correlated through the forward-kinematics computation:

$$q_j = q_{c,j} + a_j^\top \xi, \quad \xi_i \in [-1, 1],$$

where $a_j^\top$ is the j -th row of A . Affine-arithmetic propagation then produces endpoint records

$$E_k(R_{\text{obb}}) = c_k + \sum_i g_{k,i} \epsilon_i \oplus [-\rho_k, \rho_k].$$

Equivalently,

$$E_k(R_{\text{obb}}) = p_k(q_c) + J_k(q_c)A\Xi \oplus [-\rho_k, \rho_k],$$

where each coordinate remainder is bounded conservatively, for example by

$$\rho_{k,\mu} \geq \frac{1}{2} \sum_{a,b} |(A^\top H_{k,\mu} A)_{ab}|,$$

with $H_{k,\mu}$ an interval or otherwise conservative Hessian bound over R_{obb} . If these remainders are sampled rather than outward bounded, the OBB source is a performance filter and not a certificate.

OBB endpoint records feed the same endpoint-to-link construction as joint boxes. For a zonotope-plus-box endpoint record,

$$h_{E_k}(d) = d^\top c_k + \sum_i |d^\top g_{k,i}| + \sum_{\mu=1}^3 |d_\mu| \rho_{k,\mu}.$$

The zero-radius link centerline support is

$$h_\ell^\circ(d) = \max\{h_{E_a}(d), h_{E_b}(d)\},$$

and the rounded-link envelope adds the capsule radius,

$$h_\ell(d) = h_\ell^\circ(d) + r_\ell \|d\|_2.$$

Coordinate-axis queries of h_ℓ provide an AABB broad-phase, while the same support record can be used by the SupportHull/GJK narrow phase. A conservative OBB corridor edge therefore becomes another conservative region in the typed query graph, separate from the persistent dyadic LECT tree unless promoted to a scene-local graph layer.

D. Link-Envelope Representations

Endpoint records can be exported to multiple link representations without changing the upstream kinematic source. For a capsule link, $B_\ell(I) = B_\ell^c(I) \oplus \mathcal{R}(r_\ell)$, where $B_\ell^c(I)$ bounds the centerline sweep and $\mathcal{R}$ is the chosen radius lift.

a) *AABB envelope*: AABB envelopes wrap the two endpoint envelopes in one box and pad by the link radius:

$$B_\ell(I) = \text{bbox}(E_{\ell-1}(I) \cup E_\ell(I)) \oplus B_\infty(r_\ell).$$

Segment subdivision can retain a union of padded sub-envelopes for tighter filtering, but the union must remain explicit to preserve the gain.

b) *Fixed-direction slab envelope*: A k -discrete oriented polytope (k -DOP) record stores lower and upper support values along a fixed direction set. Endpoint boxes are projected onto those directions, padded by the link radius, and merged over retained segment pieces. In the runtime pipeline, these slab tests form an inexpensive intermediate filter: after the ordinary AABB rejection, non-axis directions can reject near misses before invoking GJK. They are filters unless their endpoint source and separation tests are conservative.

c) *SupportHull envelope*: SupportHull keeps the two endpoint AABBs and link radius as a compact support record. For direction d , the support point is selected from the proximal or distal endpoint box, and obstacle refinement uses a fixed-size GJK narrow phase with outward obstacle inflation. This preserves more directional structure than a single AABB while keeping the cached record compact. When present, fixed-direction slab records act only as an intermediate filter between the AABB broadphase and SupportHull/GJK refinement; they do not change the certificate condition, which still depends on conservative endpoint evidence and outward scene tests.

IV. Lifelong Envelope Cache Tree

LECT is a replay layer below the scene-level RBF cache. It stores scene-independent endpoint and link-envelope evidence keyed by robot kinematics and joint intervals. Obstacle tests, box labels, graph connectivity, and query outcomes remain scene-local; every replayed envelope must still be checked against the current obstacle set.

A. Cache Key and Split Policy

LECT is a dynamically grown binary k -dimensional (KD) tree over $\mathcal{Q}$. A node stores its joint interval, split

metadata, cached endpoint evidence, and runtime residency state. The central invariant is that evidence is *kinematics-keyed* rather than *scene-keyed*: replay imports geometry bounds, never a free-space label.

a) *Evidence/label separation*: An envelope record is replay-compatible only when the robot model, endpoint source, envelope representation, split descriptor, link radii, and active-link set match the current build. The split policy is also part of this descriptor; alternative policies are separate cache settings. For a parent interval I , let I_d^- and I_d^+ be the two children obtained by splitting joint dimension d at its midpoint. Define the cumulative downstream envelope volume as

$$\mathcal{V}(I) := \sum_{\ell} \text{vol}(B_\ell(I)).$$

One implemented schedule chooses depth-wise split dimensions that minimize the worst-child cumulative envelope volume,

$$d^* = \arg \min_d \max\{\mathcal{V}(I_d^-), \mathcal{V}(I_d^+)\}.$$

Depth-synchronous schedules keep replay deterministic; if the scheduled dimension is unavailable at a node, the runtime falls back to the widest valid dimension. Candidate dimensions are filtered by active-link reachability: splitting a joint that cannot affect any retained endpoint cannot tighten the downstream envelopes and is skipped for that descriptor.

b) *Sparse interval staging*: Deep cells need not materialize every ancestor. A planning cell can carry its exact joint interval, split counts, and descriptor fingerprint; if an exact replay record is absent, the interval is materialized on demand and then validated against the scene. Scene labels remain only in the forest or partition layer.

B. Runtime Use and Validation Boundary

LECT serves local box materialization; graph search uses the scene forest. Given a joint interval, the runtime replays or materializes endpoint evidence, derives the requested link representation, and then runs the current scene-validation policy. AABB broadphase normally precedes SupportHull/GJK refinement. Parent or child records can accelerate materialization, but no replayed record alone is a standalone free-space certificate. When a node is split, child envelopes may tighten the parent witness, but the resulting parent record is still representation-dependent and must be checked again against $\mathcal{O}$. Directional records are staged filters: a box first survives the padded AABB test, and only near survivors reach slab or SupportHull/GJK refinement. If no exact replay record exists, the interval is materialized on demand and then handled by the same validation path, so a cache miss changes cost but not semantics.

C. Storage and Warm-Build Semantics

Persistence is selective. Endpoint records and derived link-envelope records are the reusable evidence units.

IFK+AABB records can be regenerated on demand, whereas slab or SupportHull records are stored only when their tighter filters justify lookup and storage cost. Pre-warm fills target leaf records and reconstructs parents by child-hull propagation, avoiding repeated direct forward-kinematics charges at multiple depths for the same cache. In a warm build, a cache hit reconstructs the requested link envelopes and revalidates them against the current obstacles. Warm builds still regrow scene-specific boxes; persisted box labels are not imported as certificates. The registered Shelf replay is read-only during timing, so the replay store does not accumulate new evidence during those measurements. Random-scene cache growth is treated separately as part of the per-robot profile.

V. RapidBoxForest Construction

Fig. 2 illustrates the planning layer. RBF builds a scene-specific set of validated joint boxes and stores their adjacency as a typed corridor graph. This pair is the scene cache: while the obstacle set is unchanged, later queries use box membership and graph search instead of rebuilding the cover. LECT supplies reusable kinematics-keyed evidence, but every accepted scene box is still checked against the current obstacles.

A box is an axis-aligned joint interval $B = [l^{(1)}, u^{(1)}] \times \dots \times [l^{(n)}, u^{(n)}] \subseteq \mathcal{Q}$. A *validated box* is admitted by the configured scene-validation policy. When that policy is the conservative link-envelope disjointness test, the box is a free-space certificate; when a tighter performance filter is used, the box is only a planning region and any candidate path must pass query validation. Cells that do not pass validation are not inserted into the forest; selected blocked or inconclusive domains are retained for local refinement or later update.

A. Construction Objective

Given the joint-limit box $\mathcal{Q}$ and obstacle set $\mathcal{O}$, the build stage constructs

$$\mathcal{F} = \{B_1, \dots, B_N\}, \quad B_i \subseteq \mathcal{Q},$$

where each B_i satisfies the scene-validation policy. This budgeted set is concentrated around task anchors, query-relevant frontiers, and connectors. The reusable output is the pair $(\mathcal{F}, G)$, where G records adjacency and auxiliary transitions.

For boxes

$$B_i = \prod_{d=1}^n [l_i^{(d)}, u_i^{(d)}], \quad B_j = \prod_{d=1}^n [l_j^{(d)}, u_j^{(d)}],$$

RBF treats them as connected when every coordinate overlaps or is separated by at most a small tolerance ε_c :

$$\max\{l_i^{(d)}, l_j^{(d)}\} \leq \min\{u_i^{(d)}, u_j^{(d)}\} + \varepsilon_c, \quad d = 1, \dots, n.$$

Only zero-gap overlap or touching directly yields a contained box corridor. Tolerance-gap and explicit segment edges are search aids that require query validation unless they are converted into conservative box or OBB corridors.

Algorithm 1 Local box materialization around a seed.

Input: Seed q , LECT T , obstacle set $\mathcal{O}$, depth cap $D_{\max}$

Output: Validated interval or failure

```

1: start from the root interval containing  $q$ 
2: while depth  $\leq D_{\max}$  do
3:   replay or materialize endpoint and link evidence for the current
     node
4:   if the current interval passes the scene-validation policy then
5:     return current interval
6:   end if
7:   split and descend to the child containing  $q$ 
8: end while
9: return failure

```

Algorithm 2 Adaptive leaf sweep and local refinement.

Input: LECT T , obstacle set $\mathcal{O}$, sweep budget

Output: Initial forest $\mathcal{F}$ and retained collision domains $\mathcal{C}$

```

1: classify priority cells by replaying or materializing LECT evidence

2: insert cells that pass validation into  $\mathcal{F}$ ; prune covered descendants

3: retain inconclusive or blocked cells in  $\mathcal{C}$ 
4: refine only selected domains in  $\mathcal{C}$  using local materialization
5: insert refined boxes that remain inside their domain and connect
   to the forest
6: return  $\mathcal{F}, \mathcal{C}$ 

```

B. Build Pipeline

The build proceeds through four stages. First, local materialization descends LECT around a seed and returns the first interval on that descent accepted by the scene policy. Second, a leaf sweep classifies shallow cells and records blocked domains for possible refinement or later edits. Third, frontier growth adds boxes near anchors and query-relevant boundaries. Finally, consolidation merges or removes redundant boxes only when the resulting boxes still satisfy the same validation rule. Algorithms 1 and 2 define the two validation routines reused throughout this pipeline; frontier growth and consolidation repeat the same materialize–validate–connect pattern under budget limits and do not introduce a separate validation primitive.

a) Implementation controls: Validation semantics and timings are governed by the build budget, LECT descriptor, endpoint and link representation, admitted edge types, and final audit resolution. Materialization failures remain deferred domains for restricted refinement; frontier or repair witnesses count as certified only after conversion into conservative box or OBB corridors. Lower-level scheduling constants are fixed in the artifact manifest.

C. Corridor Query and Certificate

After consolidation, the forest is exported as a graph $G = (V, E_{\cap} \cup E_{\varepsilon} \cup E_s \cup E_o \cup E_{\pi})$ with one vertex per validated box. $E_{\cap}$ denotes overlap or contact edges; these are the edges that can support a contained conservative box corridor. E_{ε} denotes tolerance-gap candidates, E_s explicit segment witnesses, E_o conservative OBB transition corridors, and E_{π} compressed portal chains of conservative internal boxes. In reporting, E_{ε} , E_s , performance-mode boxes, and post-processing outside the conservative box or OBB-corridor union are accepted only after query validation.

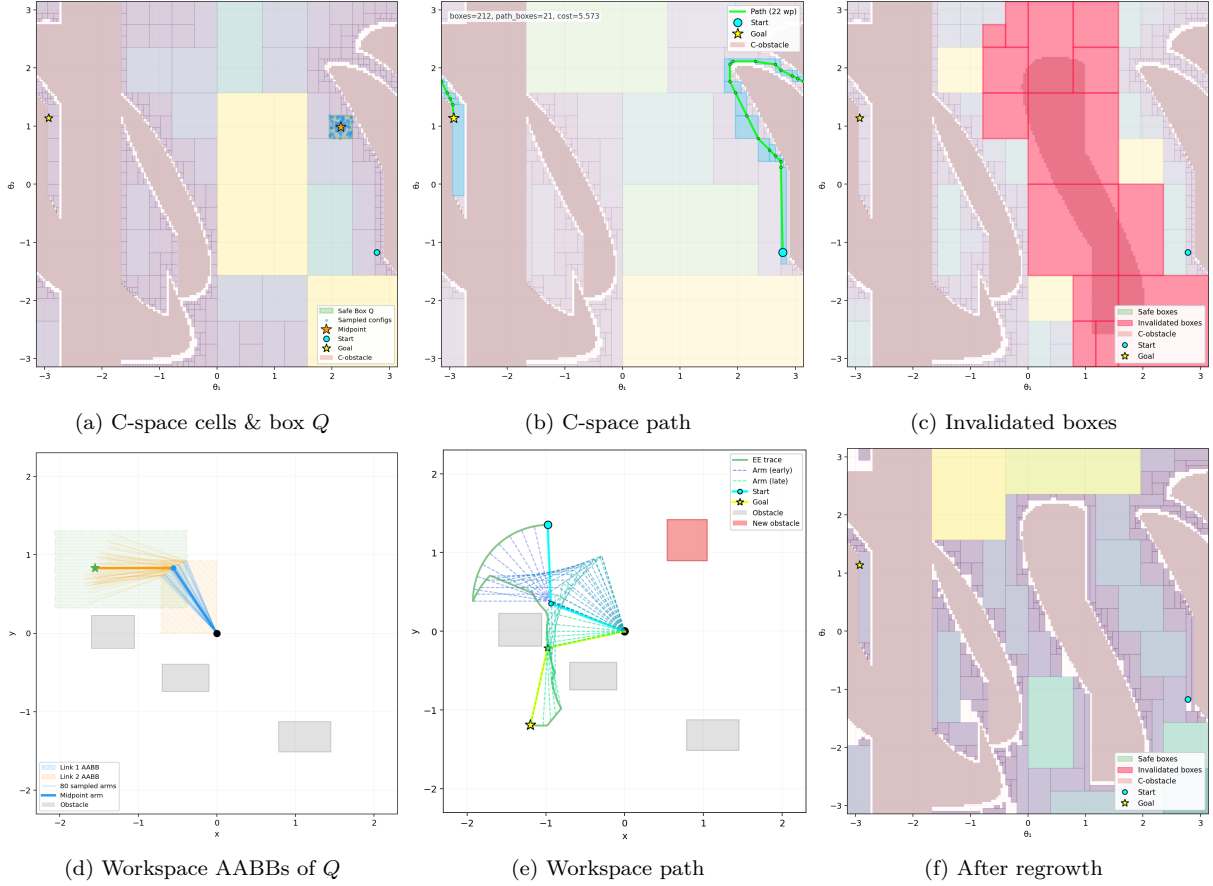


Fig. 2. Two-link schematic of the RBF build, query, and obstacle-update cycle. A highlighted C-space box maps to workspace AABBs; the box corridor yields a path; obstacle insertion invalidates intersecting boxes and triggers local refill without a full rebuild. The panels illustrate the build, query, and update logic rather than the robot geometry, workspace scale, or dimensionality of the reported experiments.

Given start and goal configurations, the query stage locates their containing or anchor boxes and searches G for a typed edge sequence. If the recovered geometric path stays inside conservative boxes, conservative OBB corridors, or expanded portal chains of conservative internal boxes, it inherits the certificate below. Otherwise it is a validated planning result, not a conservative box-corridor proof.

Theorem 1 (Conditional conservative box-corridor path). *Let $\mathcal{F}$ be a forest whose boxes have each passed conservative link-envelope validation against obstacle set $\mathcal{O}$. If a path $\gamma : [0, 1] \rightarrow \mathcal{Q}$ is continuous and satisfies*

$$\gamma([0, 1]) \subseteq \bigcup_{B \in \mathcal{F}} B,$$

then every configuration on γ is collision-free with respect to the collision model in Assumption 1.

Proof. For any $s \in [0, 1]$, the containment assumption gives a conservatively validated box $B \in \mathcal{F}$ with $\gamma(s) \in B$. By construction, validation of B means every conservative link envelope for B is disjoint from $\mathcal{O}$. Because each such envelope contains the link geometry for every configuration inside B (Corollary 1), the robot geometry at $\gamma(s)$ is also disjoint from $\mathcal{O}$. Since s was arbitrary, the whole path is collision-free. $\square$

Scope. The theorem establishes collision-freedom only for paths contained in conservatively validated regions. It does

not establish coverage or completeness: finite-budget coverage depends on anchors, depth, refinement budget, envelope tightness, and obstacle geometry. Paths using uncovered space, residual segments, smoothing, or performance-mode boxes are reported only after independent query validation.

D. Dynamic Obstacle Updates

For obstacle edits, robot kinematics and LECT records stay fixed; only scene labels and graph edges change. Deletions may promote cached blocked domains after edited-scene validation. Insertions invalidate conflicting boxes and witnesses, then run bounded local refill. The update is local by design: it maintains the represented forest and does not claim to rediscover all newly free regions after an edit.

After an update, retained or inserted boxes must satisfy the edited-scene validation policy, and typed edges must keep their original semantics. If the updated graph cannot provide a contained conservative corridor, the candidate path is accepted only after independent query validation.

VI. Experiments

The experiments evaluate the reported RBF implementation under saved scenes, fixed validation policy, and audit-separated timing. The Shelf+IIWA profile study

Algorithm 3 Obstacle deletion promotion and insertion refill.

Input: Forest $\mathcal{F}$, typed graph G , collision-domain cache $\mathcal{C}$, edit (Δ^-, Δ^+)
Output: Updated forest, typed graph, and collision-domain cache

- 1: **if** $\Delta^- \neq \emptyset$ **then**
- 2: recheck cached blocked domains affected by deleted obstacles
- 3: promote only domains that pass edited-scene validation and adjacency checks
- 4: **end if**
- 5: **if** $\Delta^+ \neq \emptyset$ **then**
- 6: remove boxes and typed witnesses hit by inserted obstacles
- 7: run a bounded local leaf sweep inside each invalidated box
- 8: insert replacement boxes that validate and connect to the surviving forest
- 9: **end if**
- 10: revalidate affected typed edges and update components
- 11: **return** updated $(\mathcal{F}, G, \mathcal{C})$

uses the KUKA LBR iiwa shelf-workcell benchmark and selects the registered RBF configuration. Cross-algorithm and random-scene rows use the same success definition, audit step, final-simplification budget, and path-length statistic unless a plotted checkpoint curve explicitly sweeps a planner budget. Dynamic-update rows report only leaf-sweep scene-cache maintenance. Random scenes and queries are saved before planner execution.

a) Protocol: Timings are wall-clock measurements on an Ubuntu 22.04 workstation with an Intel i5-12600KF CPU and 31 GiB memory. The generated tables and figures are tied to provenance manifests that record sources, thread and simplification policies, checksums, and artifact-integrity checks. The manifests record an 8-thread budget for RBF and LECT work and matching math-library thread limits; OMPL RRT-Connect, PRM, and BIT* are algorithmically single-threaded in these runners unless planner-internal parallelism is exposed. PRM and IRIS-NP/GCS (nonlinear-programming IRIS followed by GCS optimization) report reusable builds, whereas RRT-Connect and BIT* are single-query rows. These rows provide timing context under the shared audit protocol, but their reusable-state semantics are not identical. Success requires query validation with the fixed 0.01-rad segment step. Online/q denotes per-query online time and excludes final simplification and the 0.01-rad audit. Where per-query audit time (Audit/q) is present, audit-complete time is Online/q plus the fixed 0.010 s final-simplification budget and Audit/q. Numeric entries are median $[Q_1, Q_3]$, where Q_1 and Q_3 denote first and third quartiles. L/L^* uses success-only audited path references at the query level unless a caption states otherwise. The main tables are final-validation summaries, not conservative-corridor certificates, repair-free results, or parameter-sensitivity studies. Unless stated otherwise, runs check only active rounded links against $\mathcal{O}$; self-collision, attached objects, grippers, and task-specific forbidden regions require additional scene checks before using the certificate.

b) Cache and profile boundary: Warm-cache rows include compatible LECT lookup and evidence reads. Snapshot creation, loading, resident memory, and offline setup amortization are outside those rows. The label d23 denotes the registered depth-23 LECT snapshot:

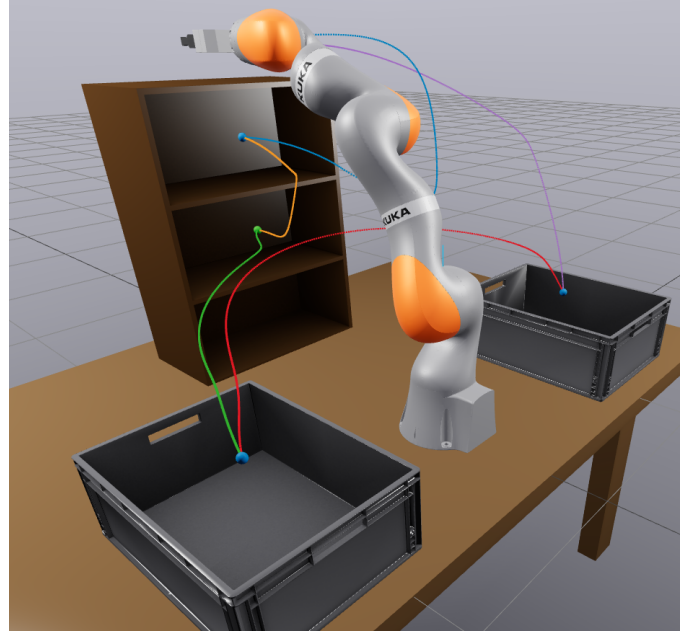


Fig. 3. Canonical Marcucci Shelf+IIWA scene with the fixed saved five-anchor query cycle.

16,777,215 records, 12.75 GiB on disk, and 958.079 s prewarm time. Unless stated otherwise, the entries in Table II replay compatible d23 evidence; the Live HIFK-5 row instead materializes HIFK-5 evidence live. The label b100 denotes the registered 100-box Shelf operating point. The baseline row uses the SupportHull representation with the standard AABB broadphase. Cache reuse requires matching robot model, endpoint source, link representation, split/depth descriptor, radii, and active-link set. Timing remains audit-separated, and bridge or repair witnesses count as certified only after conversion into conservative box or OBB corridor edges.

A. Shelf+IIWA Registered Profile Study

Fig. 3 shows the Shelf+IIWA scene and fixed five-anchor query cycle AS–TS–CS–LB–RB–AS; the anchor AS is the approach-side anchor, TS and CS are shelf-side target anchors, and LB and RB are the left and right bin anchors. Table II and Fig. 4 report the b100 registered profile over seeds 0–7. This fixed cycle is used for mechanism ablations and profile selection; the saved random catalog below tests broader start-goal variation. The table reports build time, Online/q, five-query amortized time Build/5 + Online/q, and path ratio. Critical-sample rows use the same planner and final query validation; their endpoint approximation is treated as a non-certifying filter rather than as a conservative endpoint certificate.

The registered RBF baseline builds the reusable scene structure in 0.166 [0.163, 0.170] s, has 0.130 [0.104, 0.150] s Online/q, and reaches 0.164 [0.137, 0.183] s five-query amortized time under the d23 SupportHull setting. Live HIFK-5 increases build time to 7.006 [6.947, 7.068] s because it includes live endpoint and envelope materialization. The paired rows make the cache boundary explicit: warm rows measure replay, whereas the

TABLE II
Shelf+IIWA RBF operating point and mechanism ablations that solve all saved queries.

Case	Build	Online/q	Amort. 5q	L/L^*
RBF baseline	0.166 [0.163, 0.170]	0.130 [0.104, 0.150]	0.164 [0.137, 0.183]	1.16 [1.08, 1.38]
Critical sample	0.282 [0.280, 0.285]	0.134 [0.103, 0.157]	0.190 [0.161, 0.213]	1.14 [1.08, 1.38]
Link AABB	0.141 [0.139, 0.143]	0.123 [0.099, 0.140]	0.151 [0.128, 0.168]	1.19 [1.08, 1.47]
Live HIFK-5	7.006 [6.947, 7.068]	0.234 [0.170, 0.278]	1.639 [1.576, 1.687]	1.14 [1.08, 1.39]
SupportHull only	0.349 [0.287, 0.363]	0.232 [0.155, 0.283]	0.296 [0.220, 0.352]	1.16 [1.08, 1.38]

Notes: Median $[Q_1, Q_3]$, s. Amort. 5q is Build/5 + Online/q; Online/q excludes final simplification and audit.

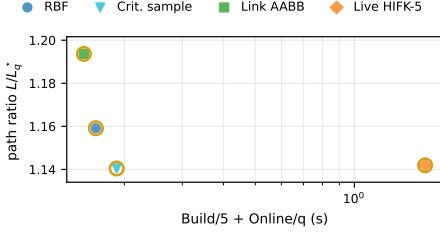


Fig. 4. Shelf+IIWA RBF mechanism trade-off. Mechanism rows that solve all saved queries are plotted at Build/5 + Online/q versus L/L_q^* ; gold rings mark the rows reported in Table II.

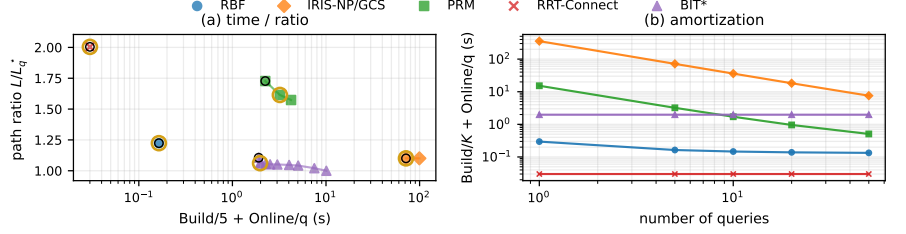


Fig. 5. Shelf+IIWA cross-algorithm trade-off. Black rings mark the first points that solve all saved queries. Timings use the Online/q column of Table III; reusable-build terms are added only for methods that report a reusable build.

TABLE III
Shelf+IIWA fixed-audit per-query cross-algorithm comparison.

Query	RBF b100 Build 0.166 s		IRIS-NP/GCS Build 355.425 s		PRM Build 20.040 s		RRT-Connect No reusable build		BIT* No reusable build	
	T (s)	L/L^*	T (s)	L/L^*	T (s)	L/L^*	T (s)	L/L^*	T (s)	L/L^*
AS→TS	0.081	1.39	0.449	1.44	0.198	1.61	0.034	2.27	4.004	1.06
	[0.078, 0.087]	[1.34, 1.48]	[0.449, 0.449]	[1.44, 1.44]	[0.147, 0.248]	[1.42, 1.73]	[0.024, 0.052]	[2.05, 2.72]	[4.002, 4.004]	[1.02, 1.08]
TS→CS	0.163	1.12	0.269	1.26	0.173	2.99	0.054	3.98	10.138	1.09
	[0.158, 0.170]	[1.09, 1.14]	[0.269, 0.269]	[1.26, 1.26]	[0.118, 0.312]	[2.11, 3.22]	[0.021, 0.089]	[2.80, 5.09]	[10.004, 10.386]	[1.04, 1.11]
CS→LB	0.109	1.47	0.381	1.07	0.136	1.49	0.032	2.20	7.504	1.03
	[0.104, 0.118]	[1.32, 1.82]	[0.381, 0.381]	[1.07, 1.07]	[0.120, 0.195]	[1.39, 2.09]	[0.027, 0.070]	[1.75, 2.51]	[7.504, 7.725]	[1.01, 1.06]
LB→RB	0.132	1.15	0.786	1.07	0.164	1.18	0.001	1.29	1.654	1.02
	[0.128, 0.141]	[1.07, 1.25]	[0.786, 0.786]	[1.07, 1.07]	[0.091, 0.399]	[1.09, 1.27]	[0.001, 0.002]	[1.27, 1.32]	[1.654, 1.654]	[1.02, 1.03]
RB→AS	0.143	1.04	0.527	1.01	0.252	1.23	0.001	1.07	0.204	1.02
	[0.138, 0.150]	[1.03, 1.05]	[0.527, 0.527]	[1.01, 1.01]	[0.096, 0.295]	[1.17, 1.24]	[0.001, 0.001]	[1.03, 1.10]	[0.204, 0.204]	[1.01, 1.03]

Notes: Median $[Q_1, Q_3]$, s. T = Online/q, excluding final simplification and audit; L/L^* uses the query-level 0.01-rad audited reference path. Single-query planners have no reusable-build term. AS denotes the approach-side anchor; TS and CS are shelf-side anchors; LB and RB are left and right bin anchors.

live row measures materialization. In the OBB-corridor profile, all 40 Shelf paths have zero residual ordinary-segment fraction after corridor conversion. Link AABB and SupportHull-without-broadphase isolate downstream representation costs: Link AABB keeps a similar path-ratio level with slightly lower online cost in this row, while skipping AABB broadphase approximately doubles amortized cost.

As a validation-boundary diagnostic, the registered pipeline returned 40/40 audited paths. An ablated variant with query bridge, repair, shortcutting, and final simplification disabled returned 0/40; this result identifies which auxiliary stages convert the reusable forest into the reported paths. The reported shelf rows are final-validated results, not conservative-corridor certificates.

B. Shelf Cross-Algorithm Context

The cross-algorithm shelf study uses the same five queries, 0.01-rad audit segment step, and 0.010 s final-

simplification budget for RBF, IRIS-NP/GCS, PRM, RRT-Connect, and BIT*. RBF uses the b100 profile and per-query Online/q measurements; PRM and IRIS-NP/GCS report reusable builds; RRT-Connect and BIT* solve each query independently. BIT* is plotted from recorded traces, so its checkpoint row is an offline quality-time reference rather than an online stopping rule. With these timing definitions, RBF's warm-cache reusable build is 0.166 s; d23 prewarm is outside that row, and snapshot load and resident memory are not included. PRM reports a 20.040 s roadmap build and IRIS-NP/GCS reports a 355.425 s reusable build, while RRT-Connect and BIT* retain lower latency on some individual queries.

C. Random Multi-Robot Scenes

The random-scene study uses saved prefix catalogs with ten queries per scene. The catalogs are generated before planner execution and reused by every method. The study

TABLE IV
Saved-catalog random-scene trade-off reference points.

Scenario	RBF			PRM			RRT-Connect		BIT*		
	Build	Online/q	L/L_q^*	Build	Online/q	L/L_q^*	Online/q	L/L_q^*	Online/q	L/L_q^*	
IIWA-Medium	0.075 [0.058, 0.083]	0.009 [0.007, 0.011]	1.00 [1.00, 1.03]	3.567 [3.007, 3.933]	0.038 [0.014, 0.070]	1.03 [1.02, 1.12]	0.002 [0.001, 0.002]	1.02 [1.01, 1.05]	0.107 [0.087, 0.154]	1.00 [1.00, 1.01]	
IIWA-Hard	0.097 [0.086, 0.105]	0.014 [0.011, 0.015]	1.00 [1.00, 1.03]	4.175 [3.408, 5.637]	0.048 [0.021, 0.102]	1.03 [1.02, 1.08]	0.002 [0.002, 0.003]	1.03 [1.00, 1.07]	0.129 [0.097, 0.186]	1.00 [1.00, 1.01]	
UR5-Medium	0.262 [0.254, 0.270]	0.075 [0.020, 0.086]	1.05 [1.00, 1.14]	3.124 [2.966, 3.418]	0.323 [0.085, 0.426]	1.02 [1.00, 1.12]	0.023 [0.002, 0.049]	1.23 [1.06, 1.54]	0.134 [0.101, 0.167]	1.01 [1.00, 1.08]	
UR5-Hard	0.276 [0.264, 0.284]	0.083 [0.043, 0.108]	1.05 [1.01, 1.14]	2.488 [2.325, 3.048]	0.419 [0.129, 0.437]	1.06 [1.01, 1.19]	0.048 [0.008, 0.090]	1.36 [1.14, 1.58]	0.154 [0.115, 0.171]	1.03 [1.00, 1.13]	
Panda-Medium	0.250 [0.218, 0.265]	0.028 [0.013, 0.039]	1.05 [1.02, 1.09]	3.431 [3.364, 4.180]	0.066 [0.038, 0.159]	1.06 [1.03, 1.08]	0.005 [0.002, 0.011]	1.22 [1.12, 1.35]	0.116 [0.092, 0.152]	1.00 [1.00, 1.00]	
Panda-Hard	0.224 [0.221, 0.237]	0.043 [0.021, 0.053]	1.05 [1.01, 1.16]	2.337 [1.627, 3.978]	0.054 [0.034, 0.126]	1.10 [1.05, 1.16]	0.056 [0.019, 0.136]	1.25 [1.14, 1.37]	0.164 [0.123, 0.214]	1.01 [1.00, 1.09]	

Notes: Median $[Q_1, Q_3]$, s. Build is a per-scene reusable-build term for RBF and PRM; RRT-Connect and BIT* are single-query rows. Online/q excludes final simplification and audit; L/L_q^* uses saved 0.01-rad audited query-level references.

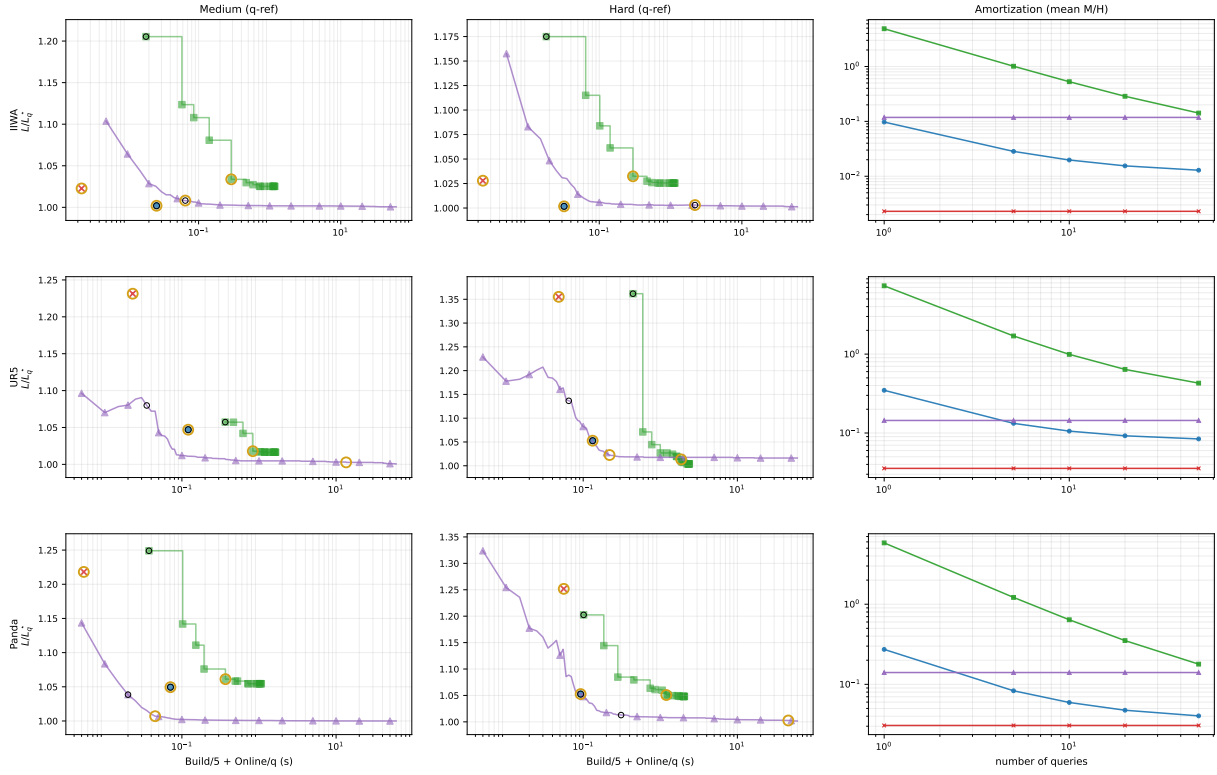


Fig. 6. Saved-catalog random-scene trade-off. Left and middle: medium and hard Build/5 + Online/q with query-level path references; right panel: Build/K + Online/q amortization over K queries. Rings mark the first checkpoints that solve all saved queries (black) and later checkpoints with at least 1% path-ratio improvement (gold). PRM and BIT* curves are offline checkpoint envelopes.

covers KUKA LBR iiwa, Universal Robots UR5, and Franka Emika Panda over medium and hard difficulties and ten seeds, giving 600 saved query evaluations per planner method. Hard scenes extend medium scenes, and trivial direct-line cases are filtered with the same fixed-audit reference probes.

Table IV and Fig. 6 summarize trade-off reference points. RBF builds one query-agnostic forest per scene and reuses it for ten queries; random rows use LECT caches and anchors tied to each robot rather than Shelf-specific assets. For RBF, each displayed row is the registered OBB-corridor profile and solves all saved queries; repair witnesses are accepted only after conversion into conservative C-space OBB corridor regions, giving zero

residual ordinary-segment fraction. PRM is built once per scene, whereas RRT-Connect and BIT* solve each query independently. The archived IRIS-NP/GCS saved-catalog run is retained as a feasibility diagnostic rather than a table row because it did not produce fixed-audit rows comparable to the other methods. Its saved-query audit outcomes were 0/100 successful hard-scene queries and 0/100 (IIWA), 30/100 (UR5), and 10/100 (Panda) successful medium-scene queries. The random-scene comparison therefore focuses on online latency under shared saved catalogs: single-query planners retain lower latency on individual queries, while RBF exposes the amortization available when the displayed forest is reused. At those operating points, RBF has lower Online/q than the BIT*

TABLE V
Endpoint-envelope source comparison over fixed joint-box widths.

Source	Volume ratio					Time (μ s)					Gap (mm)				
	0.02	0.10	0.20	0.30	0.50	0.02	0.10	0.20	0.30	0.50	0.02	0.10	0.20	0.30	0.50
IFK-AA	1.09	1.52	2.30	3.53	8.61	5.48	6.45	6.58	6.81	7.35	—	—	—	—	—
HIFK-3	1.04	1.22	1.51	1.89	3.12	25.94	29.16	27.21	29.27	29.38	—	—	—	—	—
HIFK-5	1.02	1.13	1.30	1.51	2.07	70.40	73.82	73.15	75.92	72.32	—	—	—	—	—
Critical sample	1.00	1.00	1.00	0.99	0.98	8.68	10.03	11.56	12.69	16.33	-0.05	-1.17	-4.53	-10.77	-30.97
Analytical	1.00	1.00	1.00	1.00	1.00	6429.96	6602.25	6800.10	6927.12	7096.01	0.00	0.00	0.00	0.00	-4.49
MC	0.64	0.74	0.78	0.79	0.82	2251.74	11233.26	22352.95	33813.58	56835.57	-6.05	-32.68	-65.57	-88.87	-123.53

Notes: Volumes are normalized to Analytical at each width. IFK and HIFK are the certificate-backed rows; Critical sample, Analytical, and Monte Carlo (MC) are diagnostics. Gap is the sampled-union signed gap (mm); dashes mark rows where the diagnostic is omitted.

TABLE VI
Link-envelope representation comparison over fixed joint-box widths.

Envelope	Volume						Build (μ s)						Test (μ s)					
	0.02	0.05	0.10	0.20	0.30	0.50	0.02	0.05	0.10	0.20	0.30	0.50	0.02	0.05	0.10	0.20	0.30	0.50
Link AABB	0.0136	0.0201	0.0384	0.147	0.508	4.61	0.138	0.152	0.167	0.166	0.171	0.178	0.586	0.605	0.601	0.604	0.630	0.790
SupportHull	0.000359	0.00266	0.0143	0.106	0.449	4.54	0.205	0.214	0.218	0.219	0.217	0.250	12.520	12.850	12.267	12.223	12.659	13.026

Notes: Means over fixed-width boxes. Test time pools shifted free-space probes and known-colliding obstacle probes; build time excludes endpoint time.

checkpoint rows in four of six robot-difficulty panels and lower path ratio in the two IIWA panels; BIT* has lower Online/q and path ratio for UR5-Medium and Panda-Hard.

D. Mechanism Studies

Tables V and VI isolate endpoint sources and envelope narrow phases. Table V reports endpoint volume as a ratio to the Analytical row at the same width and reports sampled-union signed gap in millimetres; negative gaps are finite-sampling diagnostics. Only IFK and HIFK rows are certificate-backed; Analytical is a reference computation for diagnostics rather than safety evidence. Table VI motivates the AABB broadphase: SupportHull/GJK is compact, but wide boxes incur high collision-test time when every obstacle reaches the GJK narrow phase.

E. Dynamic Updates

Table VII isolates scene-cache maintenance only: no query bridge, connector, simplification, audit, or query timing is included.

TABLE VII
Adaptive leaf-sweep maintenance timing.

Operation	Time (ms)	Speedup
Warm build (10 obstacles)	922.75 [897.57, 955.67]	—
Insert to 15 obstacles	6.06 [5.69, 6.54]	153.3–207.5×
Remove from 15 obstacles	0.35 [0.31, 1.65]	943.6–3354.0×
Warm build (15 obstacles)	958.79 [941.67, 1115.18]	—

Notes: Median $[Q_1, Q_3]$, ms. Maintenance only, not end-to-end replanning; speedup compares each update row with the 15-obstacle warm build.

A diagnostic that attempted audited queries directly on maintenance-only forests returned 0/8 paths for both updated and fresh forests. The table therefore measures the local maintenance primitive, not a complete replanning pipeline.

VII. Conclusion

RBF is a link-envelope-guided box-forest planner for multi-query manipulation. It uses axis-aligned boxes as

the reusable region type: endpoint-to-link swept-capsule bounds validate C-space boxes, accepted boxes form a scene-level forest and typed corridor graph, and LECT separates reusable kinematics-keyed evidence from scene labels. The certificate applies to conservative box and OBB-corridor unions; repaired paths, shortened paths, and post-processed paths require independent query validation.

Under saved catalogs, audit-separated timing, and explicit cache accounting, warm d23 Shelf replay yields a 0.166 s reusable build and 0.130 s Online/q, whereas live HIFK-5 evidence materialization increases build time to 7.006 s. In saved random scenes, RBF has lower Online/q than the BIT* checkpoint rows in four of six robot-difficulty panels at the displayed operating points; the dynamic-update rows measure only scene-cache maintenance. Within this scope, RBF is a low-overhead method for constructing reusable regions for repeated queries. Remaining limitations include conservative corridors without repair, integrated update-and-query pipelines, LECT snapshot load and memory break-even analysis, and validation for self-collision, payloads, grippers, and task-specific regions.

References

- [1] R. Deits and R. Tedrake, “Computing large convex regions of obstacle-free space through semidefinite programming,” in *Algorithmic Foundations of Robotics XI*, 2015, pp. 109–124.
- [2] T. Marcucci, M. Petersen, D. von Wrangel, and R. Tedrake, “Motion planning around obstacles with convex optimization,” *Sci. Robot.*, vol. 8, no. 84, 2023.
- [3] T. Marcucci, J. Umenberger, P. Parrilo, and R. Tedrake, “Shortest paths in graphs of convex sets,” *SIAM J. Optim.*, vol. 34, no. 1, pp. 507–532, 2024.
- [4] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006.
- [5] —, “Rapidly-exploring random trees: A new tool for path planning,” Iowa State University, Tech. Rep. TR 98-11, 1998.
- [6] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, “Probabilistic roadmaps for path planning in high-dimensional configuration spaces,” *IEEE Trans. Robot. Autom.*, vol. 12, no. 4, pp. 566–580, 1996.
- [7] S. Karaman and E. Frazzoli, “Sampling-based algorithms for optimal motion planning,” *Int. J. Robot. Res.*, vol. 30, no. 7, pp. 846–894, 2011.
- [8] A. Amice, H. Dai, P. Werner, A. Zhang, and R. Tedrake, “Finding and optimizing certified, collision-free regions in configuration space for robot manipulators,” in *Algorithmic Foundations of Robotics XV*. Springer, 2022, pp. 328–348.

- [9] M. Petersen and R. Tedrake, "Growing convex collision-free regions in configuration space using nonlinear programming," 2023. [Online]. Available: <https://arxiv.org/abs/2303.14737>
- [10] P. Wernier, A. Amice, T. Marcucci, D. Rus, and R. Tedrake, "Approximating robot configuration spaces with few convex sets using clique covers of visibility graphs," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 10359–10365.
- [11] D. Blackmore and M. C. Leu, "Analysis of swept volume via Lie groups and differential equations," *Int. J. Robot. Res.*, vol. 11, no. 6, pp. 516–537, 1992.
- [12] K. Abdel-Malek, J. Yang, D. Blackmore, and K. Joy, "Swept volumes: foundation, perspectives, and applications," *Int. J. Shape Model.*, vol. 12, no. 1, pp. 87–127, 2006.
- [13] S. Gottschalk, M. C. Lin, and D. Manocha, "OBBTree: A hierarchical structure for rapid interference detection," in *ACM SIGGRAPH*, 1996.
- [14] E. Larsen, S. Gottschalk, M. C. Lin, and D. Manocha, "Fast proximity queries with swept sphere volumes," University of North Carolina at Chapel Hill, Tech. Rep. TR99-018, 1999.
- [15] E. G. Gilbert, D. W. Johnson, and S. S. Keerthi, "A fast procedure for computing the distance between complex objects in three-dimensional space," *IEEE J. Robot. Autom.*, vol. 4, no. 2, pp. 193–203, 1988.
- [16] G. v. d. Bergen, "A fast and robust GJK implementation for collision detection of convex objects," *J. Graph. Tools*, vol. 4, no. 2, pp. 7–25, 1999.
- [17] J. Pan, S. Chitta, and D. Manocha, "FCL: A general purpose library for collision and proximity queries," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2012, pp. 3859–3866.
- [18] R. E. Moore, R. B. Kearfott, and M. J. Cloud, *Introduction to Interval Analysis*. SIAM, 2009.
- [19] L. Jaulin, M. Kieffer, O. Didrit, and E. Walter, *Applied Interval Analysis*. Springer, 2001.
- [20] J. M. Snyder, "Interval analysis for computer graphics," in *Proc. ACM SIGGRAPH*, 1992, pp. 121–130.
- [21] J.-P. Merlet, "Solving the forward kinematics of a Gough-type parallel manipulator with interval analysis," *Int. J. Robot. Res.*, vol. 23, no. 3, pp. 221–235, 2004.
- [22] —, "Interval analysis for certified numerical solution of problems in robotics," *Int. J. Appl. Math. Comput. Sci.*, vol. 19, no. 3, pp. 399–412, 2009.
- [23] S. Redon, Y. J. Kim, M. C. Lin, and D. Manocha, "Interactive and continuous collision detection for avatars in virtual environments," in *IEEE Virtual Reality Conference*, 2004, pp. 117–124.
- [24] S. Redon, M. C. Lin, D. Manocha, and Y. J. Kim, "Fast continuous collision detection for articulated models," *J. Comput. Inf. Sci. Eng.*, vol. 5, no. 2, pp. 126–137, 2005.
- [25] X. Zhang, S. Redon, M. Lee, and Y. J. Kim, "Continuous collision detection for articulated models using Taylor models and temporal culling," *ACM Trans. Graph.*, vol. 26, no. 3, p. 15, 2007.
- [26] F. Schwarzer, M. Saha, and J.-C. Latombe, "Adaptive dynamic collision checking for single and multiple articulated robots in complex environments," *IEEE Trans. Robot.*, vol. 21, no. 3, pp. 338–353, 2005.
- [27] J. A. Corrales, F. A. Candelas, and F. Torres, "Safe human–robot interaction based on dynamic sphere-swept line bounding volumes," *Robot. Comput.-Integr. Manuf.*, vol. 27, no. 1, pp. 177–185, 2011.
- [28] R. Bohlin and L. E. Kavraki, "Path planning using lazy PRM," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2000, pp. 521–528.
- [29] C. M. Dellin and S. S. Srinivasa, "A unifying formalism for shortest path problems with expensive edge evaluations via lazy best-first search over paths with edge selectors," in *International Conference on Automated Planning and Scheduling (ICAPS)*, 2016.
- [30] E. Schmerling, L. Janson, and M. Pavone, "Optimal sampling-based motion planning under differential constraints: The driftless case," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2015.
- [31] D. Coleman, I. Sucan, S. Chitta, and N. Correll, "Reducing the barrier to entry of complex robotic software: A MoveIt! case study," *J. Softw. Eng. Robot.*, vol. 5, no. 1, pp. 3–16, 2014.
- [32] T. Lozano-Pérez, "Spatial planning: A configuration space approach," *IEEE Trans. Comput.*, vol. C-32, no. 2, pp. 108–120, 1983.
- [33] R. A. Brooks and T. Lozano-Pérez, "A subdivision algorithm in configuration space for findpath with rotation," *IEEE Trans. Syst. Man Cybern.*, vol. SMC-15, no. 2, pp. 224–233, 1985.
- [34] J. J. Kuffner and S. M. LaValle, "RRT-connect: An efficient approach to single-query path planning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2000, pp. 995–1001.
- [35] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Batch informed trees (BIT*): Sampling-based optimal planning via the heuristically guided search of implicit random geometric graphs," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2015, pp. 3067–3074.
- [36] J. D. Gammell, T. D. Barfoot, and S. S. Srinivasa, "Informed sampling for asymptotically optimal path planning," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 966–984, 2018.
- [37] L. Janson, E. Schmerling, A. Clark, and M. Pavone, "Fast marching tree: A fast marching sampling-based method for optimal motion planning in many dimensions," *Int. J. Robot. Res.*, vol. 34, no. 7, pp. 883–921, 2015.
- [38] I. A. Şucan, M. Moll, and L. E. Kavraki, "The open motion planning library," *IEEE Robot. Autom. Mag.*, vol. 19, no. 4, pp. 72–82, 2012.
- [39] M. Moll, I. A. Şucan, and L. E. Kavraki, "Benchmarking motion planning algorithms: An extensible infrastructure for analysis and visualization," *IEEE Robot. Autom. Mag.*, vol. 22, no. 3, pp. 96–102, 2015.
- [40] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "CHOMP: Gradient optimization techniques for efficient motion planning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2009, pp. 489–494.
- [41] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, "STOMP: Stochastic trajectory optimization for motion planning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2011, pp. 4569–4574.
- [42] J. Schulman, Y. Duan, J. Ho, A. Lee, I. Awwal, H. Bradlow, J. Pan, S. Patil, K. Goldberg, and P. Abbeel, "Motion planning with sequential convex optimization and convex collision checking," *Int. J. Robot. Res.*, vol. 33, no. 9, pp. 1251–1270, 2014.
- [43] M. Mukadam, J. Dong, X. Yan, F. Dellaert, and B. Boots, "Continuous-time gaussian process motion planning via probabilistic inference," *Int. J. Robot. Res.*, vol. 37, no. 11, pp. 1319–1340, 2018.